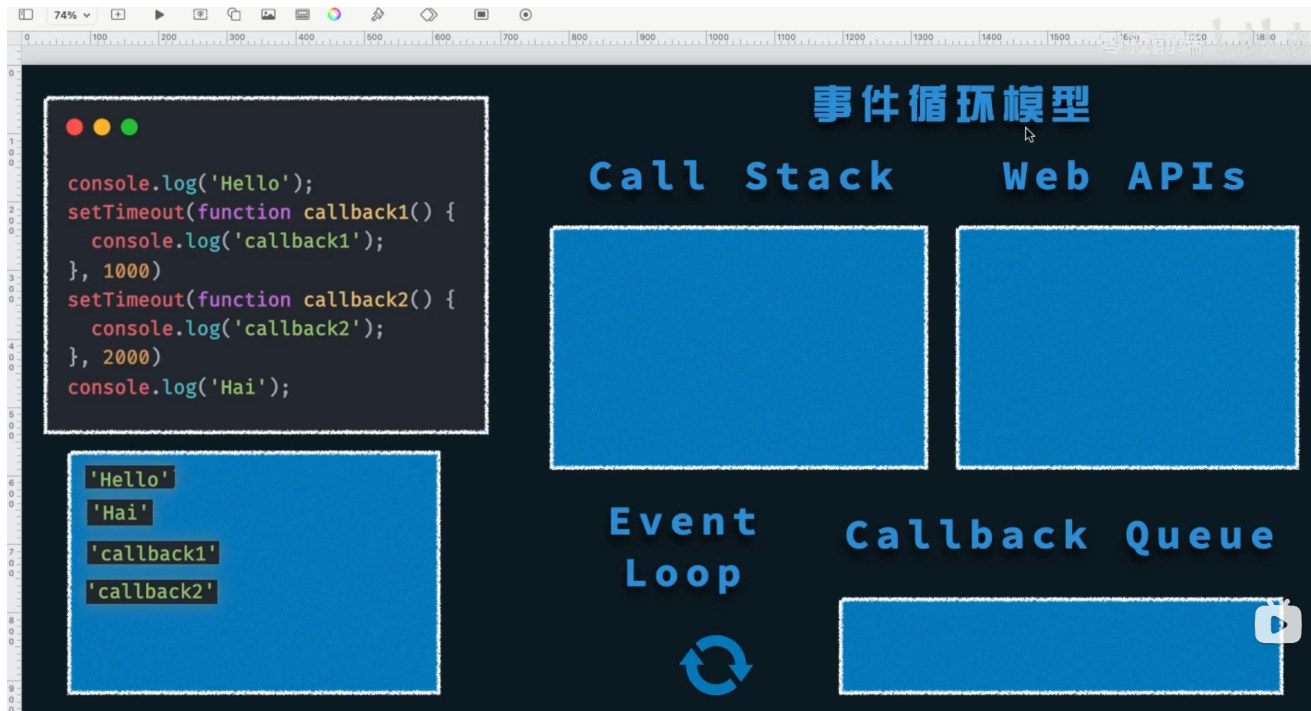


1. 事件循环：



原型链

2. js原型链

01-prototype对象

```
/*
 对象的继承：
 面向对象编程很重要的一个方面，就是对象的继承。A对象通过继承B对象，就能直接拥有B对象的所有属性和方法。这对于代码的复用是非常有用的。
*/
```

```
/*
 基于原型 (prototype) 的继承
 大部分面向对象的编程语言，都是通过“类” (class) 实现对象的继承。JS语言的继承不通过 class，而是通过“原型对象” (prototype) 实现，ES6当中有“类”的继承，但是本质还是过“原型对象” (prototype) 实现。
*/
```

```
// 各属性值不同是可以接受的，因为不同实例对象的属性值基本是不同的。可是方法却是一样的，可是方法也不能在不同实例之间共享，每个每生成一个实例对象，就要创建一个同样行为的方法，shi这样就造成了资源浪费。
```

```

<script>
    //构造函数的缺点:属性可以是不同,可以分别保存,
    //虽然两个对象的函数体完全相同,但实际上保存的是函数的地址
    //所以两个对象都各自保存了这个函数
    // 所以相同的方法却不是被p1,p2两个对象共享的,方法不能被不同的实例共享,这会造成资源的浪费,
    //
    // function Person(name, age) {

//通过prototype添加方法
function Person(name, age) {
    this.name = name;
    this.age = age;
}
Person.prototype.eat = function () {
    console.log("吃饭");
};
var P1 = new Person("xiaoming", 16);
var P2 = new Person("xiaohua", 20);
console.log(P1);
console.log(P2);
P1.eat();
P2.eat();
console.dir(Person);
//隐式原型和显示原型
console.dir(P1.__proto__);
console.dir(Person.prototype);
/*原型对象的属性和方法,都可以被实例对象所共享,不仅节省的内存,还体现了实例对象之间的联系*/

```

```

1
2 <!DOCTYPE html>
3 <html lang="en">
4   <head>
5     <meta charset="UTF-8" />
6     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7     <title>01-prototype属性</title>
8   </head>
9   <body>
10    <script>
11      //构造函数的缺点:属性可以是不同,可以分别保存,
12      //虽然两个对象的函数体完全相同,但实际上保存的是函数的地址
13      //所以两个对象都各自保存了这个函数
14      // 所以相同的方法却不是被p1,p2两个对象共享的,方法不能被不同的实例共享,这会造成资源的浪费,
15      //
16      // function Person(name, age) {
17      //   this.name = name;
18      //   this.age = age;
19      //   this.eat = function () {
20      //     console.log("吃饭");

```

```

21     //    };
22     // }
23     // var P1 = new Person("xiaoming", 16);
24     // var P2 = new Person("xiaohua", 20);
25     // console.log(P1);
26     // console.log(P2);
27
28     /*prototype属性:
29     // 每个函数都有一个prototype属性(对象才有属性, 所以相当于把函数作为一个对象来看), 指向一个
对象*/
30     // function fn() {}
31     // console.dir(fn);
32     // fn.prototype.abc = "123";
33     // console.log(fn.prototype); //对于普通函数, 此属性没什么用
34     // //确定, fn是一个对象
35     // console.log(typeof fn.prototype); //object
36
37     //通过prototype添加方法
38     function Person(name, age) {
39         this.name = name;
40         this.age = age;
41     }
42     Person.prototype.eat = function () {
43         console.log("吃饭");
44     };
45     var P1 = new Person("xiaoming", 16);
46     var P2 = new Person("xiaohua", 20);
47     console.log(P1);
48     console.log(P2);
49     P1.eat();
50     P2.eat();
51     console.dir(Person);
52     //隐式原型和显示原型
53     console.dir(P1.__proto__);
54     console.dir(Person.prototype);
55     /*原型对象的属性和方法, 都可以被实例对象所共享, 不仅节省的内存, 还体现了实例对象之间的联系
*/
56
57     //原型对象上的属性和方法, 不是实例对象自己的属性(方法)。只要修改原型对象, 变动立刻就会体
现在所有的实例对象上。
58     Person.prototype.eat = function () {
59         console.log("吃饭! ");

```

```

60     };
61     P1.eat();
62     P2.eat();
63     //在原型对象上添加属性
64     Person.prototype.position = "student";
65     console.log(P1.position, P2.position);
66
67     /*总结：
68         原型对象的作用：就是用来定义共享的属性和方法，但是一般用来共享方法。而实例对象可以看作原型对象
        衍生出来的子对象。
69     */
70     </script>
71 </body>
72 </html>
73
74

```

02-初识原型链：

```

1  <!DOCTYPE html>
2  <html lang="en">
3      <head>
4          <meta charset="UTF-8" />
5          <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6          <title>02-初始原型链</title>
7      </head>
8      <body>
9          <script>
10             /*
11                 原型链：
12                 JS规定，所有对象都有自己的原型对象，函数也是对象，所以也不例外。
13                 函数实际是由构造函数Function实例化来的
14                 1.任何对象都可以充当其他对象的原型对象
15                 2.原型对象也有自己的原型对象
16                 因此就会形成一个原型链(prototype-chain)
17             */
18             //对象上有隐式原型，又因为隐式原型全等于显式原型，所以对象是通过隐式原型来访问到函数添加到
            显式原型上的属性和方法。
19             // var fn = new Function();
20             // console.dir(fn);
21             // Function.prototype.abc = "abc";

```

```

22     // console.log(fn.abc);
23
24     function Fn() {}
25     var f1 = new Fn();
26     console.log(f1.__proto__ === Fn.prototype); //true
27     console.log(Fn.__proto__ === Function.prototype); //ture
28     console.log(Fn.prototype.__proto__ === Object.prototype); //true
29
30     //所有对象都继承了object.prototype的属性
31     console.log(Object.prototype.__proto__); //null原型链的尽头。
32
33     //Object.getPrototypeOf方法：返回参数对象的原型。
34     console.log(Object.getPrototypeOf(f1) === Fn.prototype); //true
35 </script>
36 </body>
37 </html>
38

```

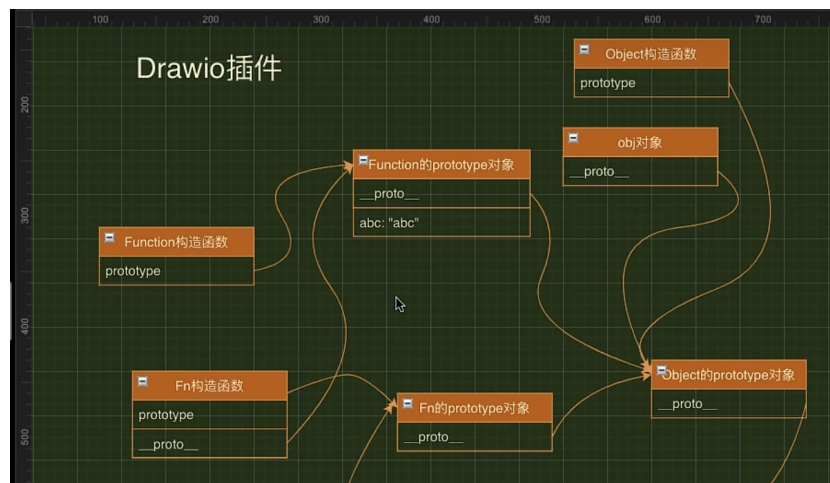
用构造函数Function实例化fn函数:

```

▼ f anonymous ( ) ⓘ
  length: 0
  name: "anonymous"
  ▶ prototype: {}
  arguments: null
  caller: null
  [[FunctionLocation]]: VM6:1
  ▶ [[Prototype]]: f ()
  ▶ [[Scopes]]: Scopes[1]

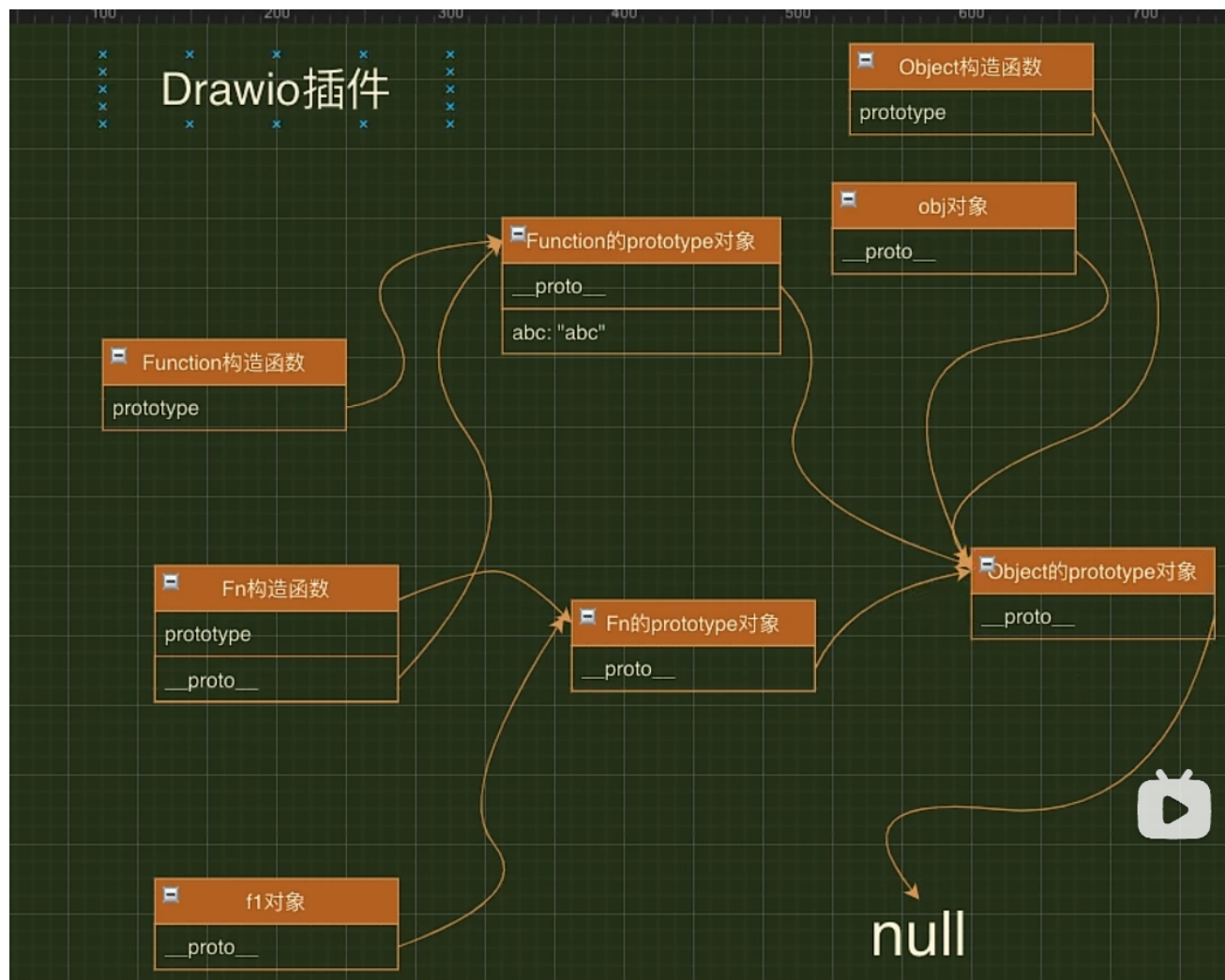
```

fn作为函数时有显式原型，作为对象时有隐式原型



fn作为对象时有隐式原型，指向Function构造函数的prototype对象，而Function构造函数有一个prototype属性，指向他的prototype对象，所以fn和Fn都指向同一个prototype地址，所以隐式原型的对象可以访问到abc属性。

原型链：



03-属性的查找过程

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>03-属性的查找过程</title>
7   </head>
8   <body>
9     <script>
10      function Fn(a) {
11        this.a = a;
12      }
13      var f1 = new Fn("a");
```



```

14     console.dir(Fn);
15     console.dir(f1);
16     console.dir(f1.a); //a
17     Fn.prototype.a = "aa";
18     console.log(f1.a); //a
19     //如果对象自身和他的原型，都定义了一个同名属性，那么优先读取对象自身的属性，这叫做覆盖
    (overriding)
20     </script>
21 </body>
22 </html>
23

```

3. new命令和this指向

this指向：

```

<script>
    /*
        new命令
        使用new命令时，它后面的函数依次执行下面的步骤。
        1. 创建一个空对象，作为将要自动返回的对象。
        2. 将这个空对象的原型指向构造函数的`prototype`属性
        3. 将空对象赋值给函数内部的`this`关键字
        4. 开始执行构造函数内部的代码
    */
    // this始终指向一个对象，是当前属性或方法所在的对象。
    // this是一个动态的值，是跟使用函数的方式有关的
    function Person() {
        this.userName = 'xiaoming';
        console.log(this);
    }
    var p1 = new Person() // 实例化构造函数生成对象，函数中的
    this就是实例化后的对象
    // console.log(p1);

    // var p2 = Person()
    var p2 = window.Person() // 没有使用关键字new，而是直接调
    用函数，相当于通过window.方法的方式执行函数。this就会指向函数
    的调用方，也就是window。
    // console.log(p2); // undefined

```

new命令：

```
<script>
// 使用new命令实例化构造函数
/* function Person() {
  this.userName = 'xiaoming'
  // return true // 如果返回的是非对象数据，则会被忽略，直接返回this对象
  return {
    userName: 'xiaohua'
  } // 如果返回的是对象，则此返回的对象会把this对象覆盖掉
}
var p1 = new Person()
console.log(p1); */

// 没有使用this的函数，如果返回的是非对象数据，返回的则是一个空对象
function Person() {
  return 1
}
var p1 = new Person()
console.log(p1);
```

// 总结：new命令总是返回一个对象，要么是实例对象，要么是函数内return语句指定的对象。

new.target: 让不是构造函数的调用，变成构造函数


```
// new.target
function Person(userName) {
  // console.log(new.target);
  if (!new.target) {
    return new Person(userName)
  }
  this.userName = userName
}

var p1 = Person('xiaoming')
console.log(p1);
var p2 = new Person('xiaohua')
console.log(p2);
```

this的由来:

函数在声明的时候，作用域就已经确定了。

为什么要设计this:

<title>04-为什么要设计this</title>

</head>

<body>

<script>

```
var a = 2
```

```
function fn() {
```

console.log(this.a); // js中一切皆对象，运行环境也是对象，所以函数都是在某个对象下运行的，this就是指函数运行时所在的对象(环境)，所以this是动态的，在调用之前是不确定的，只有在调用的时候才可以确定this的指向。

```
}
```

```
var obj = {
```

```
  a: 1,
```

```
  fn: fn
```

```
}
```

```
obj.fn()
```

```
fn()
```

```
var obj2 = {
```

```
  a: 3,
```

```
  fn: fn
```

```
}
```

```
obj2.fn()
```



4. 什么是web缓存?

web缓存是高级前端工程师必修技能。是我们变成大牛过程中绕不开的知识。

文章会尽量用通俗易懂的言语来细说web缓存的概念和用处。

本期文章的大纲是

1. 什么是web缓存（前端缓存）
2. 缓存可以解决什么问题？他的缺点是什么？
3. 强制缓存原理讲解

3.1.基于Expires字段实现的强缓存

3.2.基于Cache-control实现的强缓存

4. 协商缓存原理讲解

4.1.基于last-modified实现的协商缓存

4.2.基于ETag实现的协商缓存

什么是web缓存?

web缓存主要指的是两部分：浏览器缓存和http缓存。

其中http缓存是web缓存的核心，是最难懂的那一部分,也是最重要的那一部分。

浏览器缓存：比如,localStorage,sessionStorage,cookie等等。这些功能主要用于缓存一些必要的信息，比如用户信息。比如需要携带到后端的参数。亦或者是一些列表数据等等。

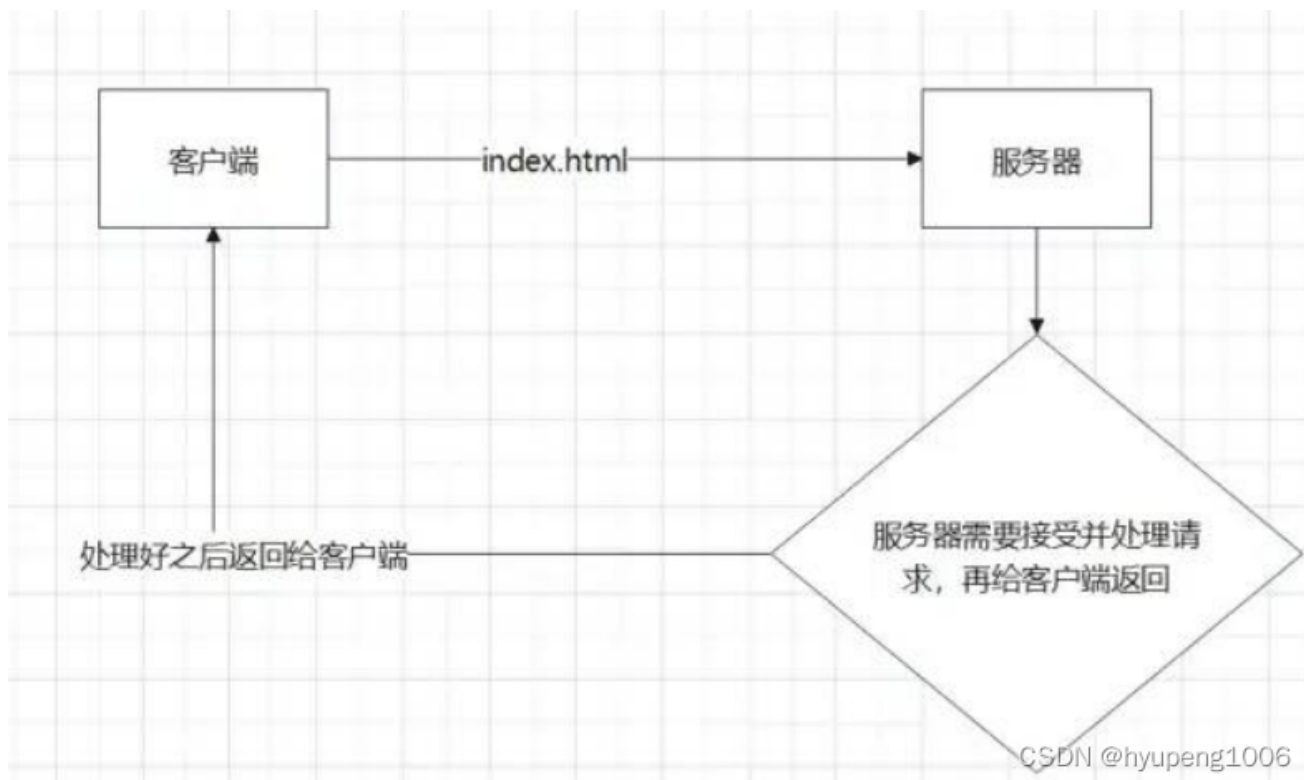
不过这里需要注意。**像localStorage, sessionStorage这种用户缓存数据的功能，他只能保存5M左右的数据，多了不行。cookie则更少，大概只能有4kb的数据。**不要担心，这些概念对于未来会称为前端大牛的你来说都不是什么问题，非常的简单。因为太简单，数据缓存不再这篇文章的介绍中，这里一笔带过

这篇文章重点讲解的是：前端[http缓存](#)。

http缓存

官方介绍:Web 缓存是可以自动保存常见文档副本的 HTTP 设备。当 Web 请求抵达缓存时，如果本地有“已缓存的”副本，就可以从本地存储设备而不是原始服务器中提取这个文档。

举个例子↓



看图，问题就是出在，服务器需要处理http的请求，并且http去传输数据，需要带宽，带宽是要钱买的啊。而我们缓存，就是为了让服务器不去处理这个请求，客户端也可以拿到数据。

注意，我们的缓存主要是针对html,css,img等静态资源，常规情况下，我们不会去缓存一些动态资源，因为缓存动态资源的话，数据的实时性就不会不太好，所以我们一般都只会去缓存一些不太容易被改变的静态资源。

缓存可以解决什么问题？他的缺点是什么？

先说说，缓存可以解决什么问题。

- 减少不必要的网络传输，节约宽带（就是省钱）
- 更快的加载页面（就是加速）
- 减少服务器负载，避免服务器过载的情况出现。（就是减载）

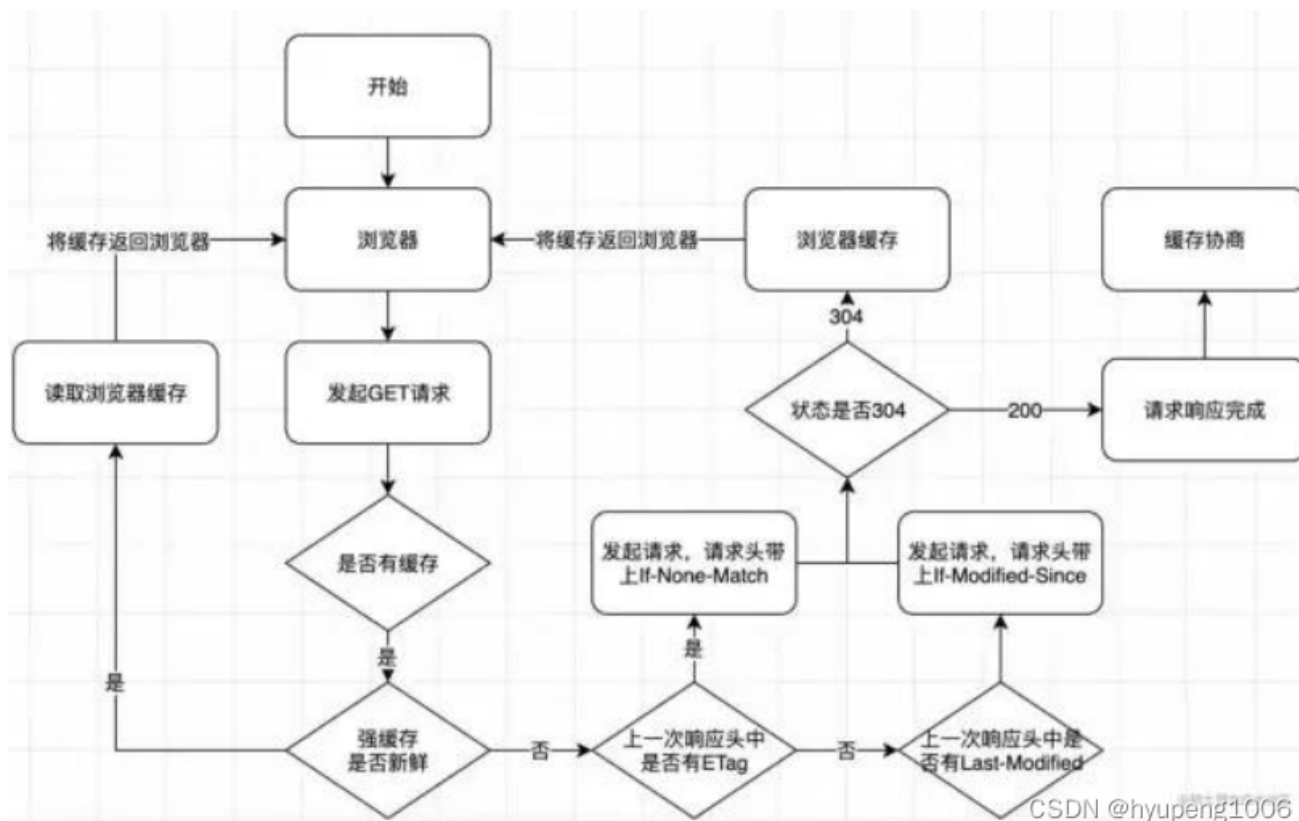
再说缺点

- 占内存（有些缓存会被存到内存中）

其实日常的开发中，我们最最最关心的，还是"更快的加载页面";尤其是对于react/vue等SPA（单页面）应用来说，首屏加载是老生常谈的问题。这个时候，缓存就显得非常重要。不需要往后端请求，直接在缓存中读取。速度上，会有显著的提升。是一种提升网站性能与用户体验的有效策略。

http缓存又分为两种两种缓存，强制缓存和协商缓存,我们来深度剖析一下强制缓存和协商缓存各自的优劣以及他们的使用场景以及使用原理

http缓存流程图↓



CSDN @hyupeng1006

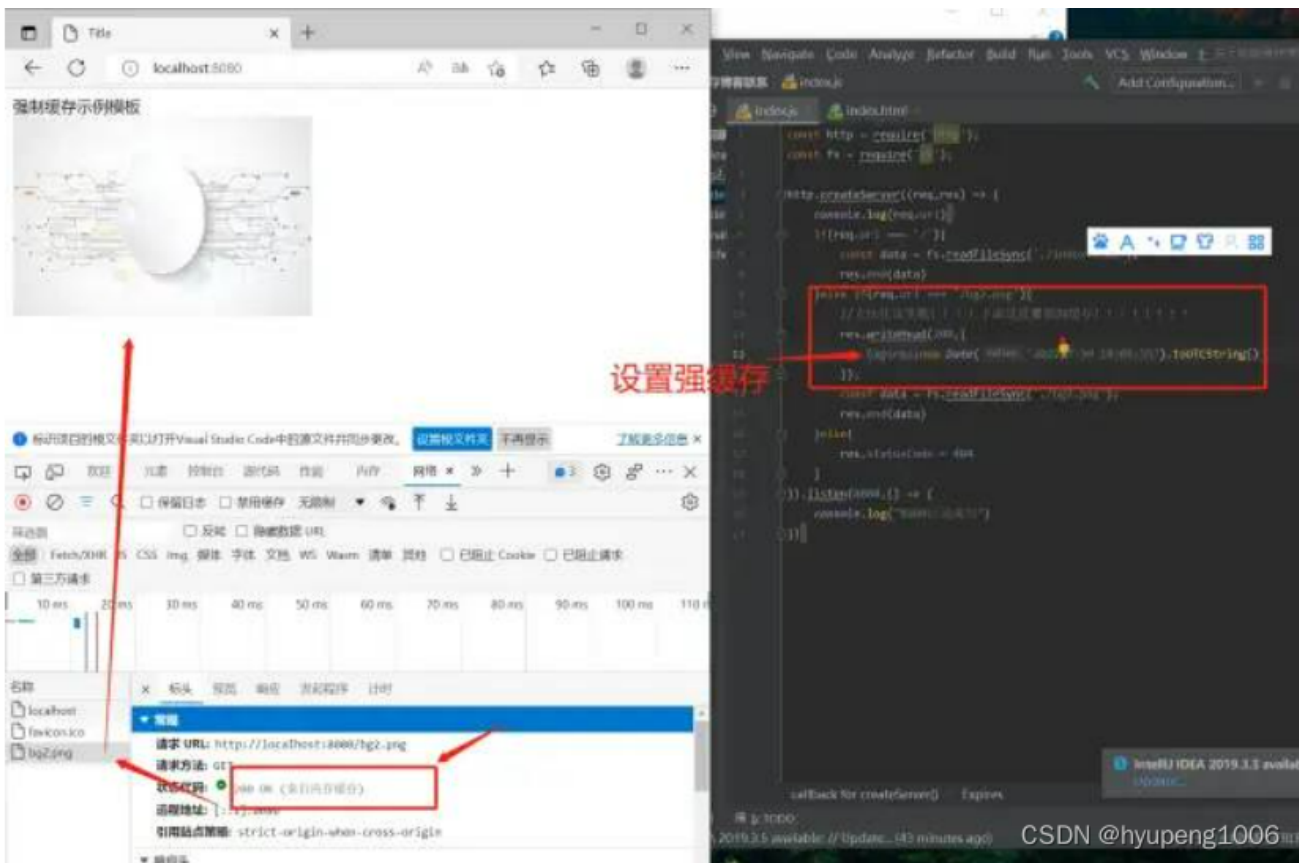
强制缓存

强制缓存，我们简称强缓存。

从强制缓存的角度触发，如果浏览器判断请求的目标资源有效命中强缓存，如果命中，则可以直接从内存中读取目标资源，无需与服务器做任何通讯。

基于Expires字段实现的强缓存

在以前，我们通常会使用响应头的 `Expires` 字段去实现强缓存。如下图↓



Expires 字段的作用是，设定一个强缓存时间。在此时间范围内，则从内存（或磁盘）中读取缓存返回。

比如说将某一资源设置响应头为:Expires:new Date("2022-7-30 23:59:59");

那么，该资源在2022-7-30 23:59:59 之前，都会去本地的磁盘（或内存）中读取，不会去服务器请求。

但是，** **Expires** 已经被废弃了**。对于强缓存来说，**Expires** 已经不是实现强缓存的首选。

因为Expires判断强缓存是否过期的机制是:获取本地时间戳，并对先前拿到的资源文件中的 **Expires** 字段的时间做比较。来判断是否需要对服务器发起请求。这里有一个巨大的漏洞：“如果我本地时间不准咋办？”

是的，**Expires** 过度依赖本地时间，如果本地与服务器时间不同步，就会出现资源无法被缓存或者资源永远被缓存的情况。所以，**Expires** 字段几乎不被使用了。现在的项目中，我们并不推荐使用 **Expires**，强缓存功能通常使用 **cache-control** 字段来代替 **Expires** 字段。

没想到吧，整半天，这个属性是废的。



哈哈哈哈哈！！！！

CSDN @hyupeng1006

基于Cache-control实现的强缓存（代替Expires的强缓存实现方法）

`Cache-control` 这个字段在http1.1中被增加，`Cache-control` 完美解决了 `Expires` 本地时间和服务器时间不同步的问题。是当下的项目中实现强缓存的最常规方法。

`Cache-control` 的使用方法很简单，只要在资源的响应头上写上需要缓存多久就好了，单位是秒。比如↓

```
1 //往响应头中写入需要缓存的时间res.writeHead(200,{ 'Cache-Control':'max-age=10'});复制代码
2 运行本项目
```

下图的意思就是，从该资源第一次返回的时候开始，往后的10秒钟内如果该资源被再次请求，则从缓存中读取。



Cache-Control: max-age=N, N就是需要缓存的秒数。从第一次请求资源的时候开始, 往后N秒内, 资源若再次请求, 则直接从磁盘(或内存中读取), 不与服务器做任何交互。

Cache-control 中因为max-age后面的值是一个滑动时间, 从服务器第一次返回该资源时开始倒计时。所以也就不需要比对客户端和服务端的时间, 解决了 **Expires** 所存在的巨大漏洞。

Cache-control 有max-age、s-maxage、no-cache、no-store、private、public这六个属性。

- **max-age**决定客户端资源被缓存多久。
- **s-maxage**决定代理服务器缓存的时长。
- **no-cache**表示是强制进行协商缓存。
- **no-store**是表示禁止任何缓存策略。
- **public**表示资源即可以被浏览器缓存也可以被代理服务器缓存。
- **private**表示资源只能被浏览器缓存。

no-cache和no-store

no_cache是 **Cache-control** 的一个属性。它并不像字面意思一样禁止缓存, 实际上, **no-cache**的意思是强制进行协商缓存。如果某一资源的 **Cache-control** 中设置了**no-cache**, 那么该资源会直接跳过强缓存的校验, 直接去服务器进行协商缓存。而**no-store**就是禁止所有的缓存策略了。

注意, no-cache和no-store是一组互斥属性, 这两个属性不能同时出现在 **Cache-Control** 中。

public和private

一般请求是从客户端直接发送到服务端, 如下↓



但有些情况下是例外的：比如，出现代理服务器，如下↓



而**public**和**private**就是决定资源是否可以在代理服务器进行缓存的属性。

其中，**public**表示资源在客户端和代理服务器都可以被缓存。

private则表示资源只能在客户端被缓存，拒绝资源在代理服务器缓存。

如果这两个属性值都没有被设置，则默认为**private**

注意，**public**和**private**也是一组互斥属性。他们两个不能同时出现在响应头的 `cache-control` 字段中。

max-age和s-maxage

max-age表示的时间资源在客户端缓存的时长，而**s-maxage**表示的是资源在代理服务器可以缓存的时长。

在一般的项目架构中**max-age**就够用。

而**s-maxage**因为是代理服务端的缓存时长，他必须和上面说的**public**属性一起使用（**public**属性表示资源可以在代理服务器中缓存）。

注意，**max-age**和**s-maxage**并不互斥。他们可以一起使用。

那么,Cache-control如何设置多个值呢？用逗号分割，如下↓

```
Cache-control:max-age=10000,s-maxage=200000,public
```

强制缓存就是以上这两种方法了。现在我们回过头来聊聊，**Expires** 难道就一点用都没有了吗？也不是，虽然

Cache-control是**Expires** 的完全替代品，但是如果要考虑向下兼容的话，在 **Cache-control** 不支持的时候，还是要使用 **Expires** ，这也是我们当前使用的这个属性的唯一理由。

协商缓存

温馨提示:协商缓存的内容会有点点绕。需要仔细阅读。

基于last-modified的协商缓存

基于last-modified的协商缓存实现方式是：

1. 首先需要在服务器端读出文件修改时间，
2. 将读出来的修改时间赋给响应头的 `last-modified` 字段。
3. 最后设置 `Cache-control:no-cache`

三步缺一不可。

如下图↓

```
const http = require('http');
const fs = require('fs');

http.createServer((req, res) => {
  console.log(req.url)
  if(req.url === '/'){
    const data = fs.readFileSync('./index.html');
    res.end(data)
  } else if(req.url === '/bg2.png'){
    //协商缓存
    const data = fs.readFileSync('./bg2.png'); //读取资源
    const {mtime} = fs.statSync('./bg2.png'); //读取修改时间
    res.setHeader('last-modified', mtime.toUTCString()); //设置文件最后修改时间
    res.setHeader('Cache-Control', 'no-cache');
    res.end(data)
  } else {
    res.statusCode = 404
  }
}).listen(8080, () => {
```

注意圈出来的三行。

第一行，读出修改时间。

第二行，给该资源响应头的 `last-modified` 字段赋值修改时间

第三行，给该资源响应头的 `Cache-Control` 字段值设置为: no-cache.(上文有介绍，Cache-control: no-cache的意思是跳过强缓存校验，直接进行协商缓存。)

还没完。到这里还无法实现协商缓存

当客户端读取到 `last-modified` 的时候，会在下次的请求标头中携带一个字段: `If-Modified-Since`。

```
Connection: keep-alive
Host: localhost:8080
If-Modified-Since: Tue, 18 Jan 2022 01:30:41 GMT
Referer: http://localhost:8080/
sec-ch-ua: " Not;A Brand";v="99", "Microsoft Edge";v="103", "Chromium";v="100"
```

而这个请求头中的 `If-Modified-Since` 就是服务器第一次修改时候给他的时间，也就是上图中的

```
res.setHeader('last-modified', mtime.toUTCString());
```

这一行。

那么之后每次对该资源的请求，都会带上 `If-Modified-Since` 这个字段，而服务端就需要拿到这个时间并再次读取该资源的修改时间，让他们两个做一个比对来决定是读取缓存还是返回新的资源。

如图↓

```

}else if(req.url === '/bg2.png'){
    //协商缓存
    const data = fs.readFileSync('./bg2.png'); //读取资源
    const {mtime} = fs.statSync('./bg2.png');//读取修改时间
    //读取第一次返回给客户端的文件修改时间
    const ifModifiedSince = req.headers['if-modified-since'];
    //比较第一次的修改时间和资源文件当前的修改时间是否一致
    if(ifModifiedSince === mtime.toUTCString()){
        //如果一致，说明文件没有被修改过，则返回304
        res.statusCode = 304;
        //因为缓存已经生效了，这里不需要返回资源data
        res.end();
        //缓存生效后直接return结束工作，避免返回新的last-modified.
        return
    }
    res.setHeader('last-modified',mtime.toUTCString());//设置文件最后修改时间
    res.setHeader('Cache-Control','no-cache');//设置协商缓存策略
    res.end(data)
}

```

CSDN @hyupeng1006

这样，就是协商缓存的所有操作了。

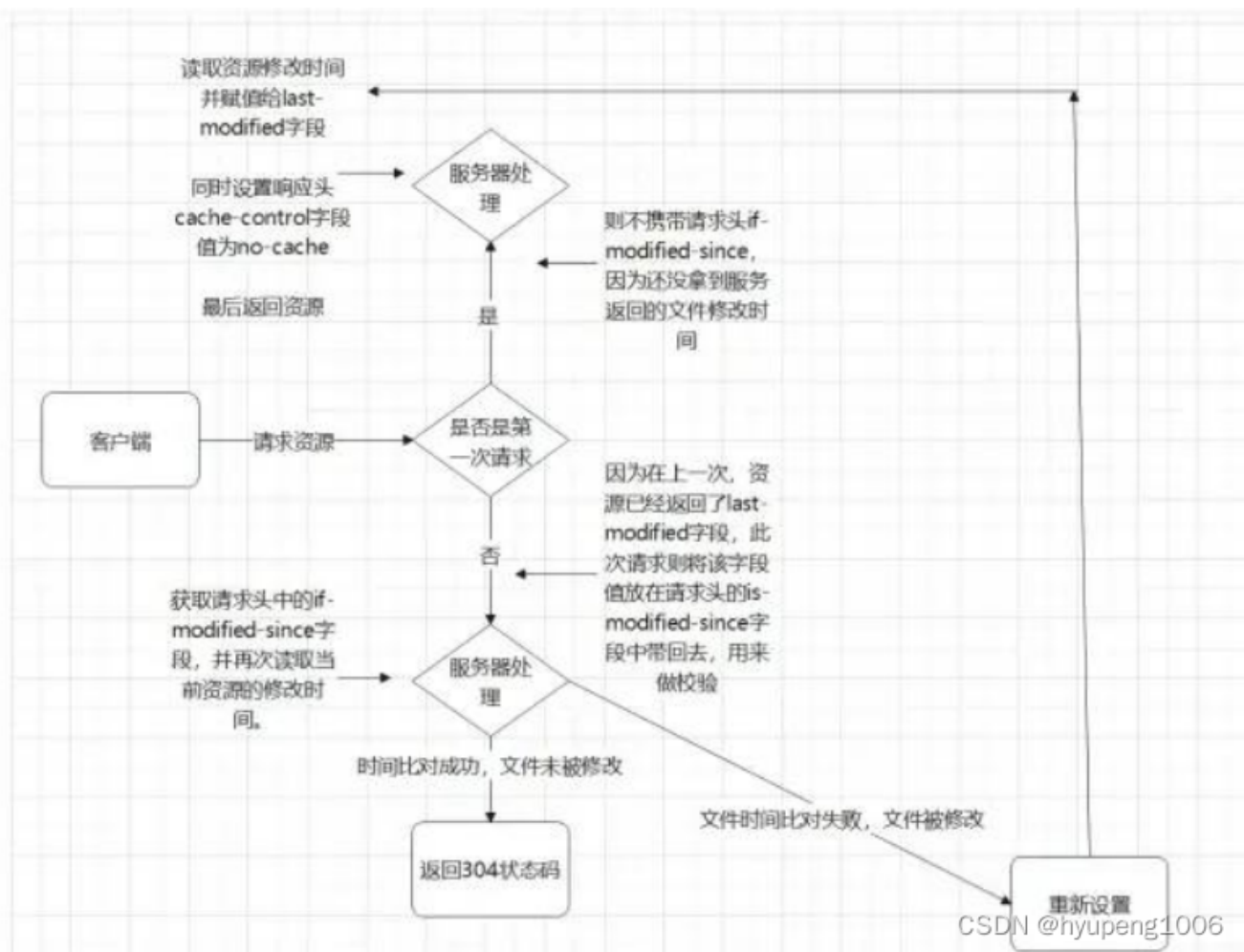
看到这里，有些小伙伴可能有些迷糊了。



纠结一番

CSDN @hyupeng1006

没关系，我们用一张图来解释下协商缓存。



使用以上方式的协商缓存已经存在两个非常明显的漏洞。这两个漏洞都是基于文件是通过比较修改时间来判断是否更改而产生的。

1. 因为是更具文件修改时间来判断的，所以，在文件内容本身不修改的情况下，依然有可能更新文件修改时间（比如修改文件名再改回来），这样，就有可能文件内容明明没有修改，但是缓存依然失效了。

2. 当文件在极短时间内完成修改的时候（比如几百毫秒）。因为文件修改时间记录的最小单位是秒，所以，如果文件在几百毫秒内完成修改的话，文件修改时间不会改变，这样，即使文件内容修改了，依然不会 返回新的文件。

为了解决上述的这两个问题。从 **http1.1** 开始新增了一个头信息，**ETag** (Entity 实体标签)

又来新东西了，兄弟们顶住



基础ETag的协商缓存

不用太担心，如果你已经理解了上面比较时间戳形式的协商缓存的话，**ETag** 对你来说不会有难度。

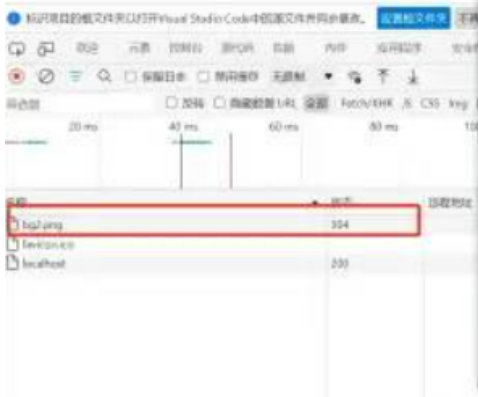
ETag 就是将原先协商缓存的比较**时间戳**的形式修改成了比较**文件指纹**。

文件指纹:根据文件内容计算出的唯一哈希值。文件内容一旦改变则指纹改变。

我们来看一下流程↓

- 1.第一次请求某资源的时候，服务端读取文件并计算出文件指纹，将文件指纹放在响应头的 **etag** 字段中跟资源一起返回给客户端。
- 2.第二次请求某资源的时候，客户端自动从缓存中读取上一次服务端返回的 **ETag** 也就是文件指纹。并赋给请求头的 **if-None-Match** 字段，让上一次的文件指纹跟随请求一起回到服务端。
- 3.服务端拿到请求头中的 **is-None-Match** 字段值（也就是上一次的文件指纹），并再次读取目标资源并生成文件指纹，两个指纹做对比。如果两个文件指纹完全吻合，说明文件没有被改变，则直接返回304状态码和一个空的响应体并return。如果两个文件指纹不吻合，则说明文件被更改，那么将新的文件指纹重新存储到响应头的 **ETag** 中并返回给客户端

代码图例↓



```

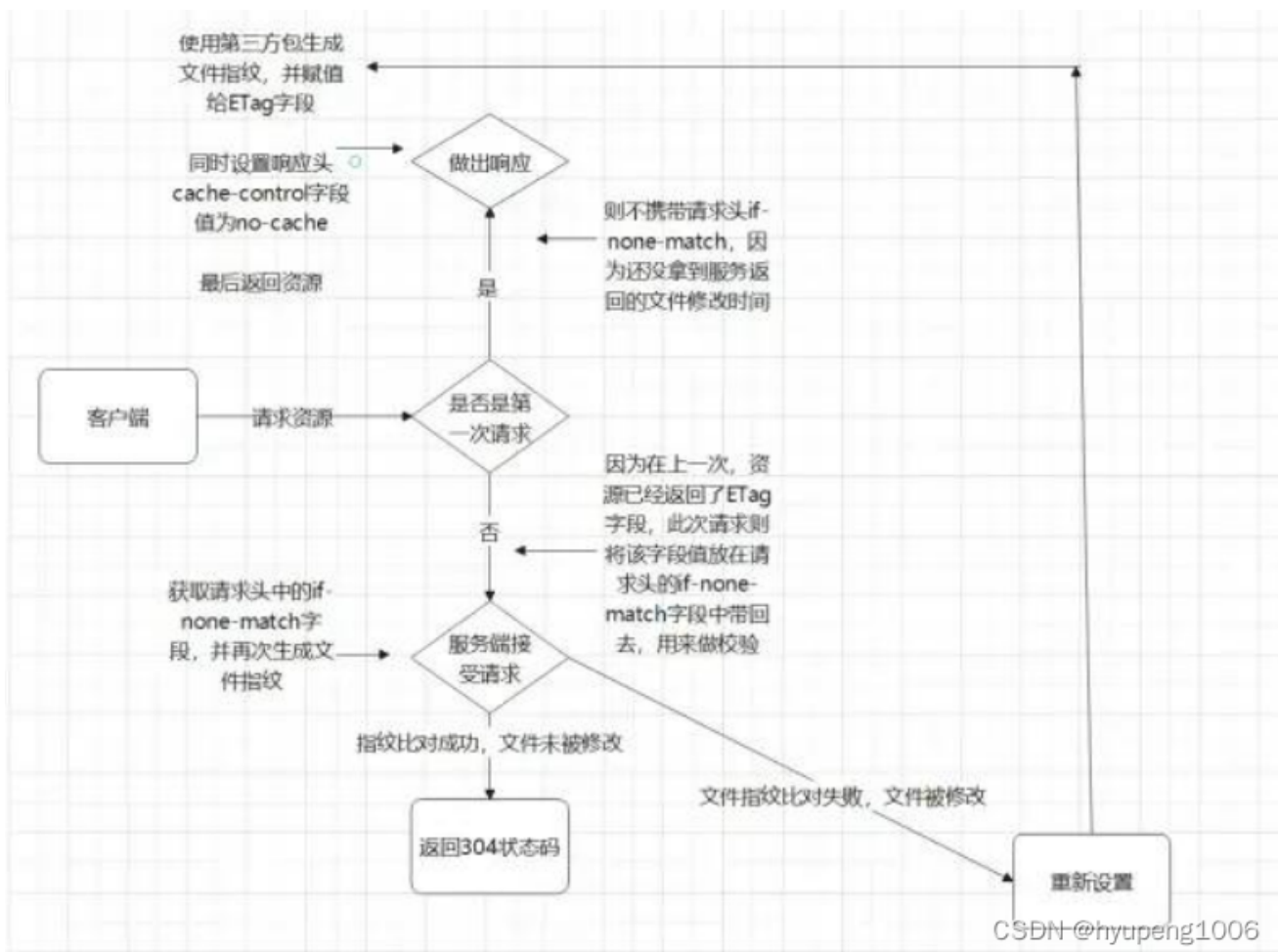
const http = require('http');
const fs = require('fs');
const etag = require('etag');

http.createServer((req, res) => {
  if (req.url === '/') {
    const data = fs.readFileSync('index.html');
    res.writeHead(200, {
      'Content-Type': 'text/html'
    });
    res.end(data);
  } else if (req.url === '/img.png') {
    // 读取图片文件
    const data = fs.readFileSync('img.png');
    const etagHeader = etag(data); // 生成 ETag 值
    const ifNoneMatch = req.headers['if-none-match']; // 获取请求头中的 if-none-match 值
    // 检查是否携带 if-none-match 头
    if (ifNoneMatch === etagHeader) {
      // 携带 if-none-match 头且值与服务器返回的 ETag 值匹配
      res.writeHead(304, {
        'ETag': etagHeader
      });
      res.end();
    } else {
      // 未携带 if-none-match 头或值不匹配
      res.writeHead(200, {
        'ETag': etagHeader
      });
      res.end(data);
    }
  }
})(req, res) => {
  // ... (other code)
});

```

CSDN @hyupeng1006

流程示例图↓



CSDN @hyupeng1006

从校验流程上来说，协商缓存的修改时间比对和文件指纹比对，几乎是一样的。

ETag也有缺点

- ETag需要计算文件指纹这意味着，服务端需要更多的计算开销。。如果文件尺寸大，数量多，并且计算频繁，那么ETag的计算就会影响服务器的性能。显然，ETag在这样的场景下就不是很适合。
- ETag有强验证和弱验证，所谓将强验证，ETag生成的哈希码深入到每个字节。哪怕文件中只有一个字节改变了，也会生成不同的哈希值，它可以保证文件内容绝对的不变。但是，强验证非常消耗计算量。ETag还有一个弱验证，弱验证是提取文件的部分属性来生成哈希值。因为不必精确到每个字节，所以他的整体速度会比强验证快，但是准确率不高。会降低协商缓存的有效性。

值得注意的是，不同于 `cache-control` 是 `expires` 的完全替代方案(说人话:能用 `cache-control` 就不要用 `expires`)。 `ETag` 并不是 `last-modified` 的完全替代方案。而是 `last-modified` 的补充方案 (说人话: 项目中到底是用 `ETag` 还是 `last-modified` 完全取决于业务场景，这两个没有谁更好谁更坏)。

追加

有掘友说👉



57coder 前端 Java 餐馆炒菜员

3小时前

文章大篇幅介绍了缓存是什么，有哪些字段，代表什么含义，却没有说我们怎么设置缓存，哪些文件适合对应的缓存类型 😊

👍 1 💬 1

删除



工边页字 (作者)

2小时前

哈哈哈，是我的疏忽，我晚点加一下。👉

CSDN @hyupeng1006

我来补足一下。

如何设置缓存

从前端的角度来说:

你什么都不用干，缓存是缓存在前端，但实际上代码是后端的同学来写的。如果你需要实现前端缓存的话啊，通知后端的同学加响应头就好了。

从后端的角度来说

请参考文章，虽然文章里的后端是使用node.js写的，但我写了详细的注释。对于后端的同学来说。应该不难看懂。

哪些文件对应哪些缓存

这个，我确实忘了说。哈哈哈。

有哈希值的文件设置强缓存即可。没有哈希值的文件（比如index.html）设置协商缓存

为什么有哈希值的文件设置强缓存

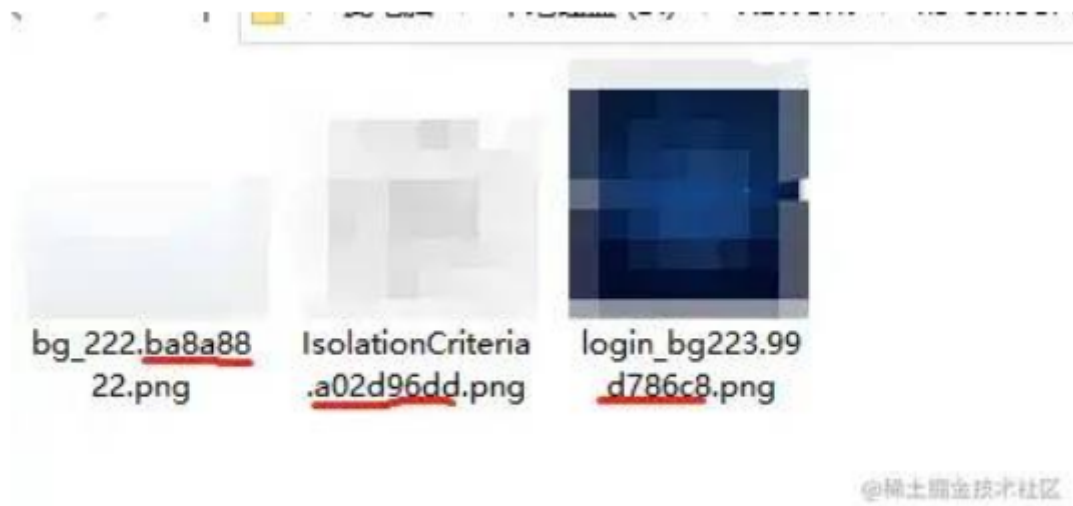
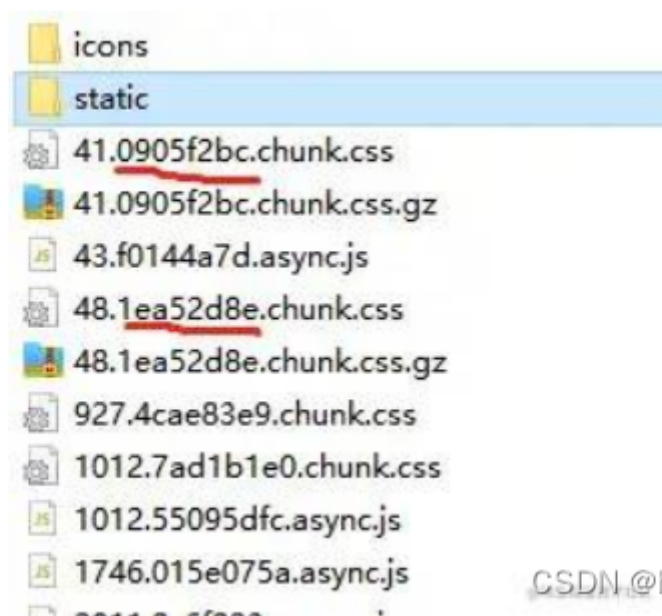


image.png



CSDN @hyupeng1006

这是我打完包之后的css文件。大家是否注意到。我划了红线的部分。明显，这绝不是我的文件名。这串和乱码一样的字符串叫 **哈希值**。每次打包之后都会生产一串新的 **哈希值** 并追加到我们的文件名中。哈希值是打包后的文件名的一部分。

我们给css设置强缓存，哪怕缓存1W年。只要我们重新打包，生产新的哈希值。那么文件名就更改了。对于机器来说，更改了文件名的文件，就是一个新的文件。

举个例子👉

比如，有一个css文件a1

第一次打包a1.css文件追加哈希值变成了 a1.aaaaa.css,我们给a1.aaaaa.css设置了强缓存1W年。

然后项目改动，我们又打包了一次。打包后生产新的哈希值，a1.aaaaa.css变成了a1.bbbbbb.css文件。那么当我们第一次访问a1.bbbbbb.css文件的时候是不会被缓存。因为1W年的缓存是给a1.aaaaa.css文件做的。关我a1.bbbbbb.css文件什么事？这样我们也就能拿到最新的改动。

其他可以被webpack生成哈希值的文件同理。

为什么index.html使用协商缓存

既然img/css这些文件都可以用强缓存。通过更改文件名的方式来获取最新的数据，为什么我堂堂index.html就要用协商缓存呢？

我给大家看个图

home_bg.png	2022/8/10 18:
index.html	2022/8/10 18:
IsolationCriteria.png	2022/8/10 18:
layouts_BasicLayout.cbe4bd92.chunk...	2022/8/10 18:
layouts_BasicLayout.cbe4bd92.chunk...	2022/8/10 18:
layouts_BasicLayout.e25e9e0c.async.js	2022/8/10 18:
layouts_BasicLayout.e25e9e0c.async....	2022/8/10 18:
layouts_BlankLayout.e98fcb14.async.js	2022/8/10 18:
layouts_SecurityLayout.aef9dc76.asy...	2022/8/10 18:
layouts_HeartLayout.26640607.chunk...	2022/8/10 18:

因为一般情况下，index.html是不会设置哈希值的。（具体得看自己项目下的dist文件夹）

注意：哈希值是需要webpack生成的。不是天生的。不过有些框架会自带（比如我使用的umi.js），设置缓存前务必看下自己的dist文件。因为如果没有配置的话，你可能所有文件都不带哈希值。

总结一下

- http缓存可以减少宽带流量，加快响应速度。
- 关于强缓存，`cache-control` 是 `Expires` 的完全替代方案，在可以使用 `cache-control` 的情况下不要使用 `expires`
- 关于协商缓存，`etag` 并不是 `last-modified` 的完全替代方案，而是补充方案，具体用哪一个，取决于业务场景。
- 有些缓存是从磁盘读取，有些缓存是从内存读取，有什么区别？答：从内存读取的缓存更快。
- 所有带304的资源都是协商缓存，所有标注（从内存中读取/从磁盘中读取）的资源都是强缓存。

5. 图解网络

图解网络介绍 | 小林coding | Java面试学习

6. we安全xss

7.

8. 1

9.

